Solution: Balanced Binary Tree (easy)

Problem Statement

Determine if a binary tree is height-balanced.

A binary tree is considered height-balanced if, for each node, the difference in height between its left and right subtrees is no more than one.

Examples:

1. **Input**:

```
   3
  / \
 9  20
   / \
  15  7
```

- **Expected Output**: true
- **Justification**: Every node in the tree has a left and right subtree depth difference of either 0 or 1.

2. **Input**:

```
      1
     / \
    2   2
   /     \
  3       3
 / \     / \
4   4   4   4
```

- **Expected Output**: true
- **Justification**: Each node in the tree has a left and right subtree depth difference of either 0 or 1.

3. **Input**:

```
      1
     / \
    2   5
   /
  3
 /
4
```

- **Expected Output**: false
- **Justification**: The root node has a left subtree depth of 3 and right subtree depth of 1. The difference (3 - 1 = 2) exceeds 1, hence the tree is not balanced.

Solution

To check if a binary tree is balanced, an efficient approach involves a depth-first traversal of the tree. Starting from the root, recursively compute the height of both the left and right subtrees. If at any node, the height difference between its left and right subtree exceeds one, the tree is imbalanced. Otherwise, proceed to check its child nodes. The height of a node is the maximum height of its left or right subtree plus one. This way, we can efficiently traverse each node once and check the height difference of the left and right subtree.

1. **Base Case**: If the node is null, return a depth of 0.
2. **Recursive Case**: Recursively calculate the depth of the left and right subtrees. If the difference in these depths exceeds 1, or if any recursive call indicates the tree is unbalanced, mark the tree as unbalanced.
3. Return the depth of the current subtree (1 plus the maximum of the depths of the left and right subtrees).

One key observation is that if any subtree is unbalanced, the entire tree is unbalanced. Thus, if at any step we discover an unbalanced subtree, we can stop our check and return false.

Algorithm Walkthrough:

For the tree:

```
      1
     / \
    2   5
   /
  3
 /
4
```

1. Starting at the root node, which is `1`:
   - We'll evaluate both the left and right subtrees.
2. For its left child (`2`):
   - We'll examine both its left and right subtrees.
3. For the left child's left child (`3`):
   - It has a left child, `4`, with no further descendants. The maximum depth from this node is `1`.
   - It doesn't have a right child, so its depth is `0`.
   - Difference in depths for node `3`: 1 - 0 = 1 (This is within the allowable range of ≤ 1).
4. The left child (`2`) of the root doesn't have a right child. Thus, its depth is `0`.
5. For the root's left child (`2`):
   - The maximum depth for its left subtree (from node `3` to `4`) is `2`.
   - It doesn't have a right subtree, so its depth is `0`.
   - Difference in depths for node `2`: 2 - 0 = 2 (This difference is greater than 1, signifying that this subtree is unbalanced).
6. For the root's right child (`5`):
   - It doesn't have a left child or a right child, which means both depths are `0`.
7. Comparing both children of the root node (`1`):
   - The left subtree has a maximum depth of `3`.
   - The right subtree has a depth of `1`.
   - Difference in depths for node `1`: 3 - 1 = 2 (This exceeds the threshold of 1).
8. Considering the findings from steps 5 and 7, the algorithm determines that the tree is **not balanced.**

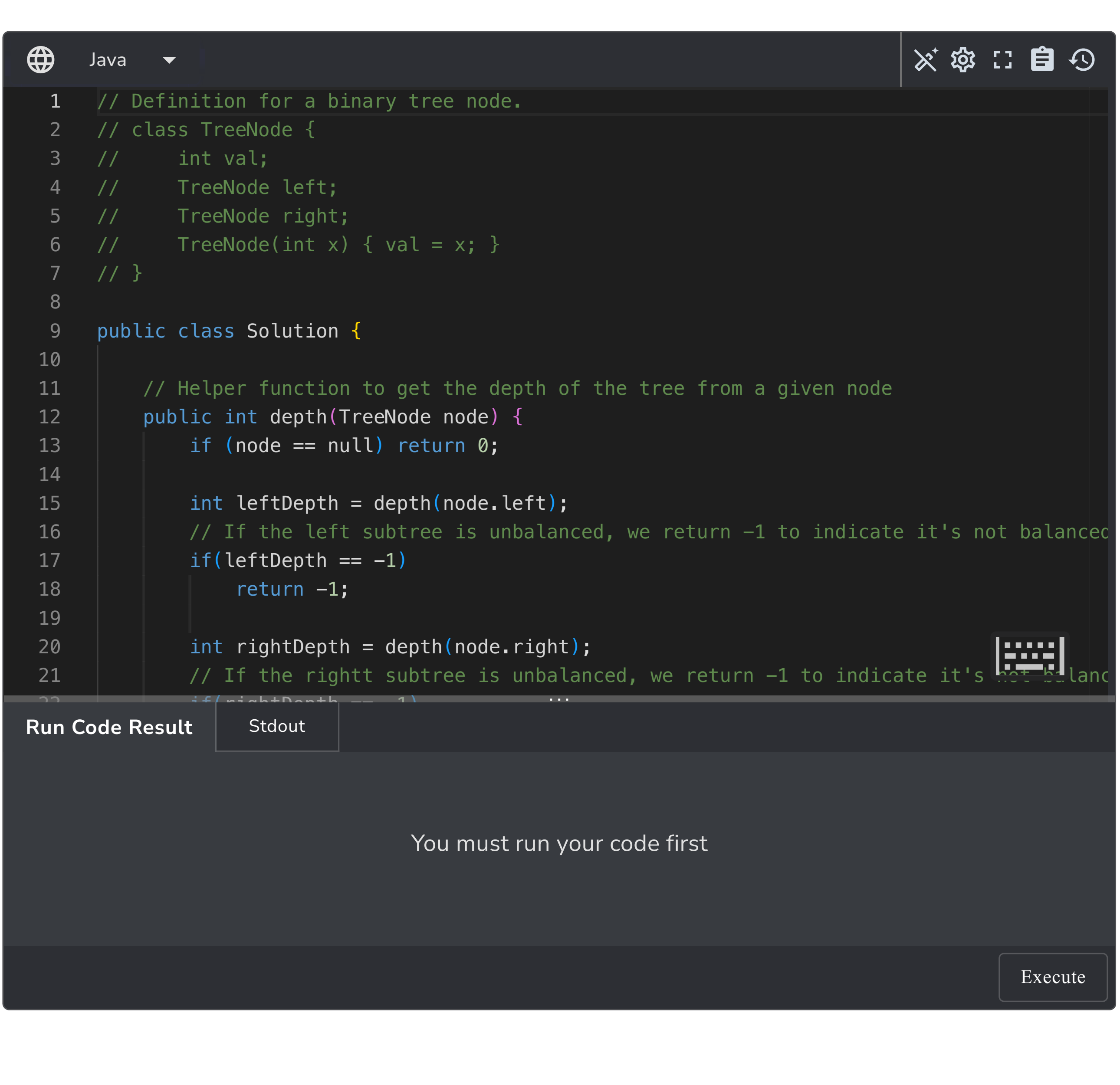
Here is the visual representation of the algorithm:

LeftDepth: ∞
RightDepth: ∞
LeftDepth: ∞
RightDepth: ∞
LeftDepth: 0
RightDepth: ∞
NULL
0 will be returned as the node is NULL
LeftDepth: 0
RightDepth: 0
Since Math.Absolute(LeftDepth - RightDepth) = 0 which is not > 1 so we will return the depth of the subtree i.e 1
0 will be returned as the node is NULL
Now, we will traverse back and will reach to the right of the node 3
LeftDepth: ∞
RightDepth: ∞
LeftDepth: ∞
RightDepth: ∞
LeftDepth: 0
RightDepth: 0
Since Math.Absolute(LeftDepth - RightDepth) = 0 which is not > 1 so we will return the depth of the subtree i.e 1
0 will be returned for left and righr depth as both nodes are NULL
Now, we will reach to the node 7 and from node 7, we will first go to its left and then right. Hence, 1 will be returned i.e. Max height
LeftDepth: 0
RightDepth: 0
Now, we will traverse back and will reach node 20
LeftDepth: 1
RightDepth: 1
Since Math.Absolute(LeftDepth - RightDepth) = 0 which is not > 1 so we will return the depth of the subtree i.e 2
Now, we will traverse back and will reach node 3
LeftDepth: 1
RightDepth: 2
Since Math.Absolute(LeftDepth - RightDepth) = 1 which is not > 1 so we will return the depth of the subtree i.e 3
Since, -1 is not returned and it shows tree is balanced
Now, we will consider a new example
LeftDepth: ∞
RightDepth: ∞
LeftDepth: 0
RightDepth: 0
Since, subtree is balanced so the max height will be returned i.e. 1
LeftDepth: 1
RightDepth: 0
Since, subtree is balanced so the max height will be returned i.e. 2
LeftDepth: 2
RightDepth: 0
Since Math.Absolute(LeftDepth - RightDepth) = 2 which is > 1 so we will return -1
Since, -1 is returned and it shows tree is not balanced

Balanced Binary Tree

Code

Here is the code for this algorithm:



```java
// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {

    // Helper function to get the depth of the tree from a given node
    public int depth(TreeNode node) {
        if (node == null) return 0;

        int leftDepth = depth(node.left);
        // If the left subtree is unbalanced, we return -1 to indicate it's not balance
        if(leftDepth == -1)
            return -1;

        int rightDepth = depth(node.right);
        // If the rightt subtree is unbalanced, we return -1 to indicate it's not balanc
```

Complexity Analysis

Time Complexity: O(n), where n is the number of nodes in the tree. Each node is visited once.

Space Complexity: O(h), where h is the height of the tree. This accounts for the maximum depth of the recursive call stack.

← Previous
Balanced Binary Tree (Easy)
Next →
Minimum Difference Between BST Nodes (Easy)
☐ Mark as Completed